FPT UNIVERSITY

TOPIC: MUSIC STREAMING PLAYLIST MANAGER

Course: CSD201

Lecturer: Đỗ Đức Hào

Group 3

Nguyễn Duy Hoàng	SE201697
Nguyễn Thành Sơn	SE201777
Nhiêu Sĩ Luân	SE201956
Bùi Văn Nghĩa	SE201912

Ho Chi Minh City, 24/05/2026

Semester: SP_2026

REPORT 1 4
1. Data Structure Decomposition 4
2. Functional Object Decomposition 4
3. Pattern Recognition in Data Organization 5
3.1 Core Data Organization Patterns 5
3.2 Behavioral Data Flow Patterns 6
3.3. Dynamic Interaction through Sequence Diagrams 6
4. Flowchart 7
4.1. Flowchart: Next Song and Previous Song 8
5. Mind Map 8
6. Research Questions (RQ) 9
7. Initial Sketch vs AI Suggestion 9
REPORT 2 10
8. Abstraction 10
8.1. Package Structure 10
8.2. Class Diagram 10
8.3. Main Classes and Responsibilities 11
8.4. Rationale for the Selected Data Structures 12
9. Playback Workflow 12
10. DSA Design: Algorithm Design 13
10.1 Shuffle Playlist - Fisher-Yates Algorithm 13
10.2. Play Next Track 14
10.3. Previous Track 14
10.4. Library Search with HashMap + ArrayList 14
11. Algorithm Trace 14
12. AI Audit & Adaptation Log 15
REPORT 3 15
13. UML Modeling of the Finalized Architectural Solution 15
13.1 Architectural View 15
13.2 UML Class Diagram 16
13.3 Layered Component Diagram 18
13.4 Sequence Diagram: Next Track 18
13.5 Sequence Diagram: Previous Track 19
13.6 Sequence Diagram: Add Song to Playlist 19
13.7 Sequence Diagram: Shuffle Playlist 20
14. Conceptual Framework 20

14.1 Framework Components 21
14.2 Research Hypotheses 21
14.3 Experimental Variables 21
14.4 Complexity Summary from the Current Source Code 22
15. Experiment Design for Algorithm Performance Testing 22
15.1 Experiment Environment 22
15.2 Dataset Generation 23
15.3 General Benchmark Procedure 23
15.4 Individual Experiments 23
15.5 Empirical Results 24
Discussion 25

REPORT 1

1. Data Structure Decomposition

- **Song Entity (Data Node - Encapsulation):**
  - Instead of being a simple struct, Song is modeled as an object that stores all metadata of a music track, including id, title, artist, duration, and filePath.
  - Its attributes are kept private to protect data integrity and are accessed through getter/setter methods. Each Song object also stores a next reference, allowing it to act as a node inside the CircularLinkedList.
- **CircularLinkedList Entity (Repeat Support):**
  - CircularLinkedList is an independent class that manages the playlist structure through head, tail, and size.
  - It provides core operations such as addLast(), remove(), isEmpty(), and clear(). These methods allow the playlist to add songs, remove songs, check empty status, and reset the list.
  - Its main advantage is that the last node always points back to the head node. Therefore, when the user presses Next at the end of the playlist, playback can continue automatically and smoothly from the first song.
- **HistoryStack Entity (Previous Support):**
  - HistoryStack manages playback history using the LIFO principle: Last-In-First-Out.
  - The class exposes simple operations such as push(), pop(), peek(), isEmpty(), getSize(), and clear(). Internally, it uses StackNode to store song references.
  - This structure is suitable for the Previous feature because the most recently played song should be restored first when the user goes back.
- **ArrayList Entity (Shuffle and Library Traversal):**
  - A linked list is inefficient for random access because it must traverse nodes sequentially with O(n) time complexity.
  - Therefore, ArrayList<Song> is used for shuffleList inside PlaylistManager. This allows the system to store song references and apply Fisher-Yates shuffle more efficiently.
  - ArrayList<Song> is also used as songList inside PlaybackController to store the music library for display and title-based search.
- **HashMap Entity (Library Lookup by ID):**
  - In this structure, each song id is used as the key, and the corresponding Song object is stored as the value.
  - This design allows getSongById(id) to retrieve a selected song with average O(1) time complexity, which is more suitable for the current web application because PlayerServlet receives songId from the user interface.

2. Functional Object Decomposition

The system is divided into interacting objects and follows the Single Responsibility Principle (SRP). Each class focuses on one main responsibility so that the system is easier to maintain and extend.

- **Data Management Module (PlaylistManager):**
  - **Responsibility:** Handles the integrity and operations of the playlist data structure.
  - **Behavior:** Acts as a wrapper around CircularLinkedList. It is responsible for adding new songs, removing songs, checking whether the playlist is empty, and maintaining the shuffleList used for Shuffle mode. This class does not control whether a song is currently playing or paused.

- **Playback Control Module (PlaybackController):**
  - **Responsibility:** Works as the central coordinator of the music player. This class owns and coordinates PlaylistManager, HistoryStack, HashMap<String, Song> songMap, and ArrayList<Song> songList.
  - **Behavior:** Handles the main playback navigation logic.
    ▪ When nextTrack() is called, it retrieves the next song from either the CircularLinkedList or the shuffled ArrayList, depending on the isShuffle state. It also uses HistoryStack.push() to save the current song before changing to the next song.
    ▪ When prevTrack() is called, it uses HistoryStack.pop() to restore the most recently played song.
    ▪ It controls state variables such as isShuffle, isRepeat, and currentShuffleIndex to decide how the playback flow should move between linked-list traversal and array-based shuffle traversal.
    ▪ For library lookup, it uses HashMap<String, Song> to retrieve a song quickly by id and ArrayList<Song> to store and search songs by title.
- **Web Interface / Controller Module:**
  - **Responsibility:** Communicates with the user through the web interface instead of a console-based interface.
  - **Behavior:** The user interacts with the web page by clicking actions such as Play, Next, Previous, or Search. These requests are handled by servlet controllers such as PlayerServlet and LibraryServlet. The controllers call PlaybackController, receive the processed result, and return data back to the web page for display.

3. Pattern Recognition in Data Organization

Pattern Recognition helps identify suitable data structures and workflow patterns for each function of the Music Streaming Playlist Manager. After replacing BST with HashMap and ArrayList, the core patterns remain the same for playback, while the library search pattern becomes simpler and more suitable for the current web demo.

3.1 Core Data Organization Patterns

Cyclic Sequential Pattern

- Problem: Repeat All requires the playlist to move continuously from one song to the next without stopping at the final node.
- Applied pattern: Circular Linked List. The tail node points back to the head node, so nextTrack() can move forward naturally.

- Benefit: next traversal is O(1), the boundary check is reduced, and the playback flow remains continuous.

LIFO / Undo Pattern

- Problem: Previous should return to the actual song heard before, not simply move backward in the physical playlist order. This is especially important when Shuffle is enabled.
- Applied pattern: HistoryStack. The controller pushes the current song before playing a new one and pops the latest song when Previous is selected.
- Benefit: push() and pop() are O(1), and the history remains correct even if the shuffle order is different from the original playlist order.

Random Access Pattern

- Problem: Shuffle needs fast access to songs by index. A linked list is not suitable for direct random access because it must traverse nodes sequentially.
- Applied pattern: ArrayList<Song> shuffleList. PlaylistManager stores playlist song references in an ArrayList and shuffles them with Fisher-Yates.
- Benefit: after the shuffle order is prepared, each shuffled next operation can read the next index in O(1).

Key-Value Lookup Pattern

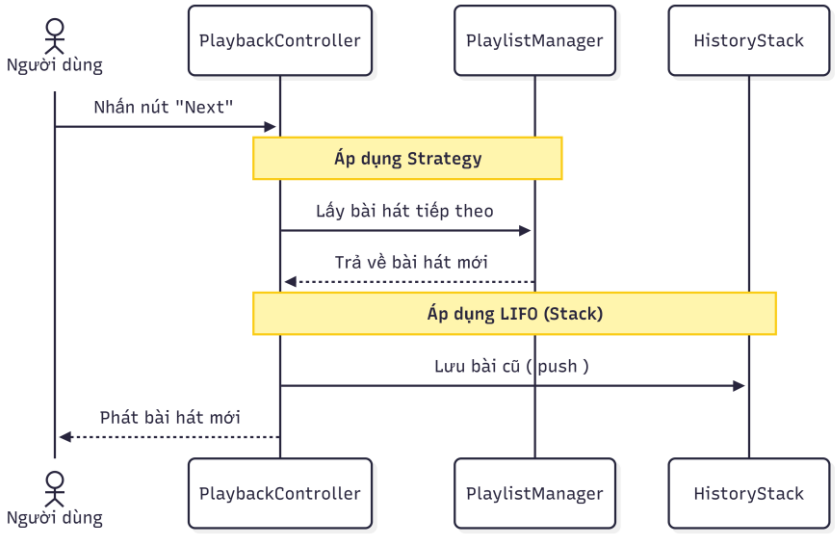
- Problem: The web interface sends songId when the user clicks Play. The system must retrieve the selected Song quickly and reliably.
- Applied pattern: HashMap<String, Song> songMap. Each song id is the key and the Song object is the value.
- Benefit: getSongById(id) has average O(1) lookup time and is easier to integrate with the current servlet workflow than the previous BST approach.

3.2 Behavioral Data Flow Patterns

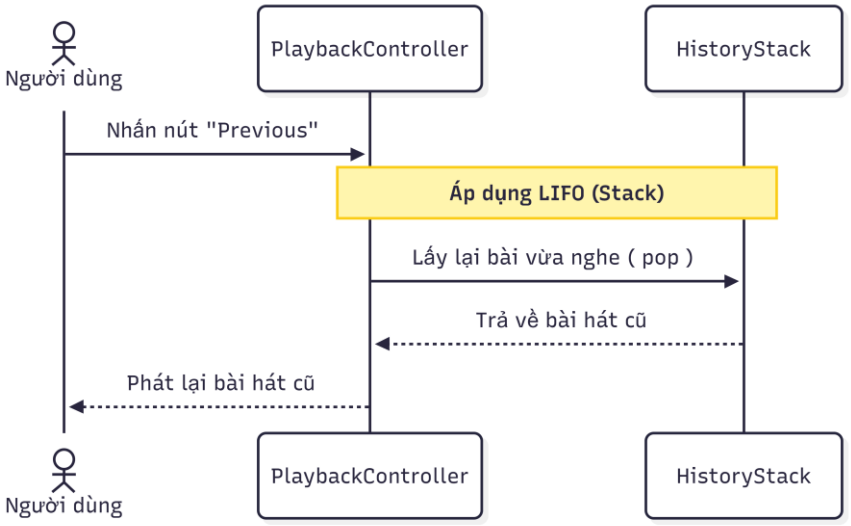
- Repeat Playlist - Cyclic Traversal: the current pointer moves through CircularLinkedList and naturally loops from tail to head.
- Previous Song - LIFO Recovery: the system restores the latest actual playback state from HistoryStack.
- Shuffle Song - Array-Based Traversal: the system reads songs from shuffleList using currentShuffleIndex and reshuffles when one shuffle cycle ends.
- Library Search - HashMap + ArrayList: song id lookup uses HashMap, while title search uses a linear scan over songList with contains() after converting to lowercase.

3.3. Dynamic Interaction through Sequence Diagrams

Sequence Diagram 1: Next Track. The user sends the Next command to PlaybackController. The controller decides whether to use shuffleList or CircularLinkedList, then calls playedSong(nextSong). The old current song is pushed into HistoryStack before the new song becomes currentPlayingSong.

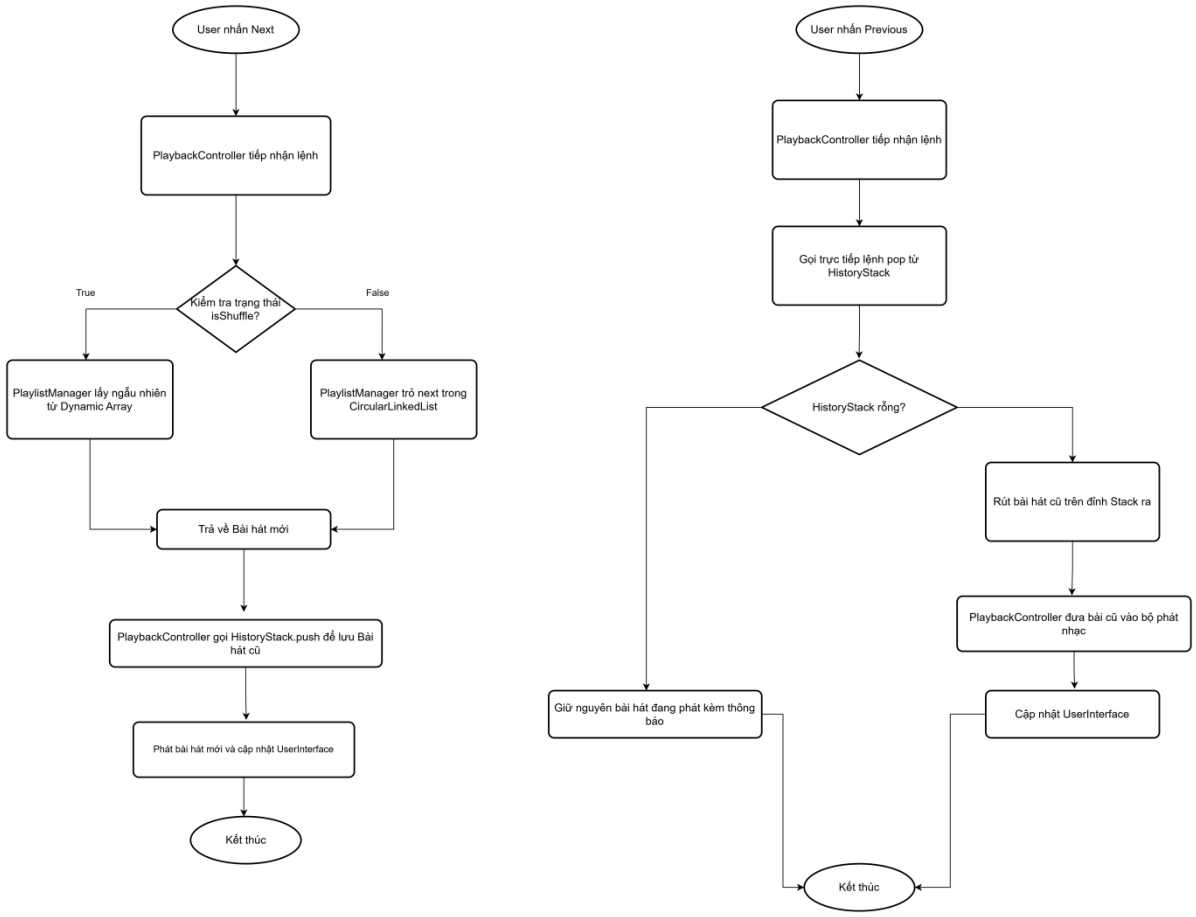


Sequence Diagram 2: Previous Track. The user sends the Previous command. PlaybackController checks HistoryStack and directly pops the latest song. It updates currentPlayingSong without calling playedSong() again, preventing duplicate history entries.



4. Flowchart

The flowchart describes the two main playback operations: Next Song and Previous Song. The logic is kept close to the original report. The main content update is that shuffle uses ArrayList, and library selection uses HashMap/ArrayList instead of BST.



4.1. Flowchart: Next Song and Previous Song

Flowchart Next Song:

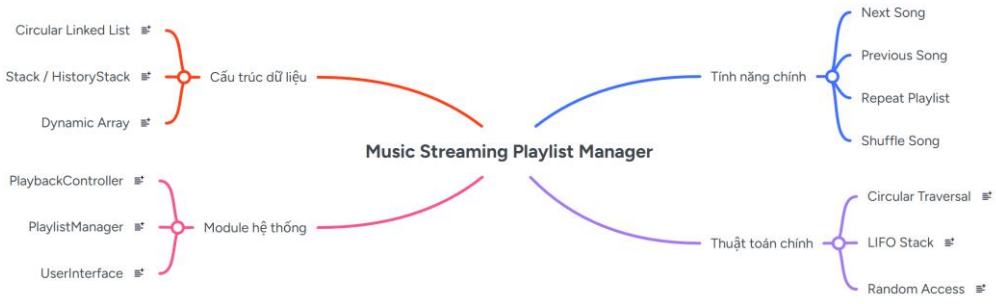
- The flow starts when the user clicks Next.
- PlaybackController receives the command and checks isShuffle.
- If isShuffle is true, PlaylistManager gets the next song from shuffleList, which is an ArrayList.
- If isShuffle is false, PlaylistManager follows the next pointer in CircularLinkedList.
- The selected song is passed to playedSong(), the previous current song is pushed into HistoryStack, and the user interface is updated.

Flowchart Previous Song:

- The flow starts when the user clicks Previous.
- PlaybackController calls pop() from HistoryStack.
- If the stack is empty, the system keeps the current state and returns idle/null status.
- If the stack is not empty, the top song is popped and becomes currentPlayingSong.

5. Mind Map

The mind map summarizes the system through four main branches: main features, data structures, algorithms, and system modules. It has been updated to include HashMap for ID lookup and ArrayList for library/search/shuffle support.



6. Research Questions (RQ)

6.1. RQ1: Optimizing cyclic navigation and playback history

- **Research question:** *How does the combination of Circular Linked List and Stack optimize continuous playlist navigation and preserve accurate playback history when users perform Next and Previous actions?*
- **Explanation:** CircularLinkedList allows Next to run in a loop with O(1) traversal. HistoryStack preserves the real playback order, so Previous remains correct even when the shuffle mode changes the physical sequence of songs.

6.2. RQ2: Handling shuffle performance and library lookup after replacing BST

- **Research question:** *How can the system support shuffle, title search, and direct song selection without using a BST?*
- **Explanation:** The current source code uses ArrayList<Song> for shuffle and title search, and HashMap<String, Song> for direct ID lookup. This combination is simple, fast for songId-based selection, and appropriate for the current in-memory web application demo.

7. Initial Sketch vs AI Suggestion

Initial Idea	AI Suggestion	Final Decision	Reason
Use a normal doubly linked list. Use manual if-else conditions to check list boundaries.	Upgrade to a Circular Linked List.	The group kept the original design direction and accepted the Circular Linked List approach.	The circular structure allows Next and Repeat All to run continuously. It also removes unnecessary boundary checks and makes the playlist traversal cleaner.
The group proposed using ArrayList to access songs randomly by index.	Directly shuffle the linked nodes using Fisher-Yates, or repeatedly convert data	The group rejected the initial AI suggestions. Instead, the group added ArrayList<Song> /	Directly shuffling linked nodes or converting data repeatedly wastes resources and may increase complexity to

	between Linked List and a temporary array.	shuffleList in PlaylistManager to store song references.	O(n). ArrayList supports faster indexed access and is more suitable for Shuffle mode.
The group planned to move backward by using pointers directly on the main playlist.	AI suggested two options: 1. Use a static array to store played songs. 2. Use an independent Stack.	The group kept the original design direction but rejected option 1 because a static array limits history storage. The group accepted option 2: using Stack.	Stack follows the LIFO principle, which accurately preserves playback history. This is especially important when Shuffle mode is enabled because the actual listening order may differ from the playlist order.

REPORT 2

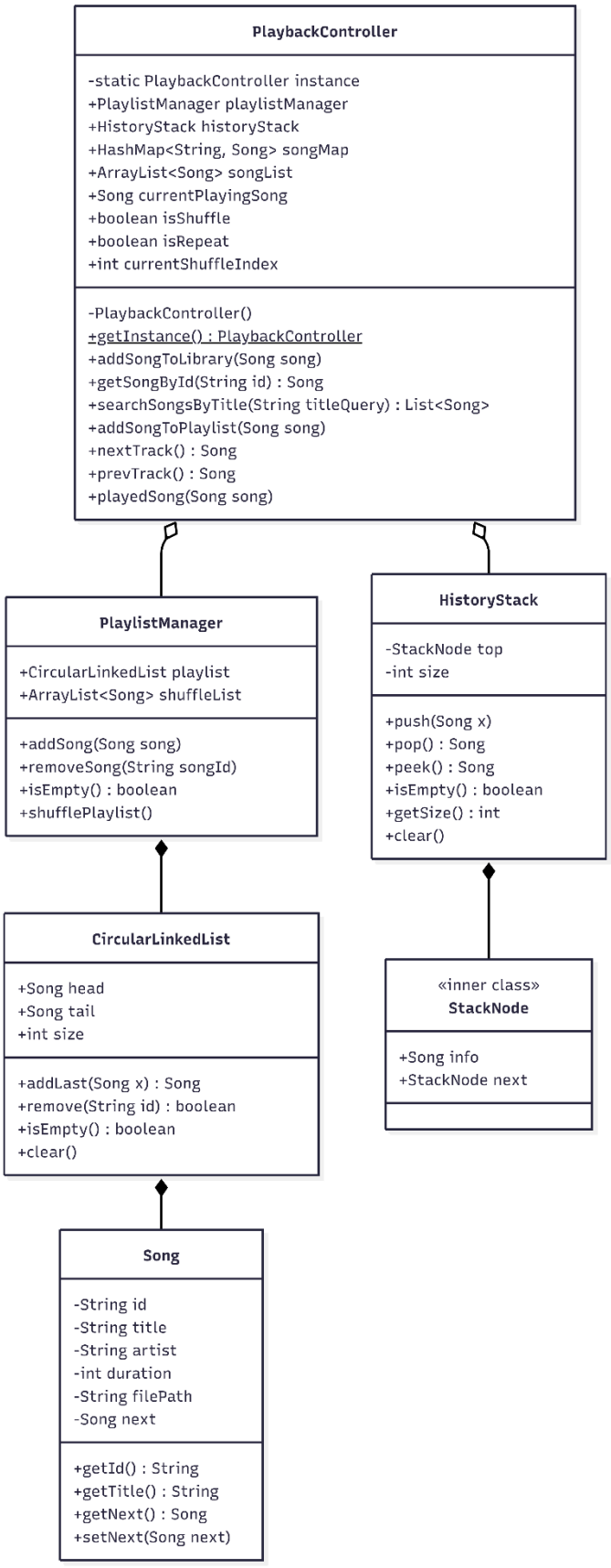
8. Abstraction

8.1. Package Structure

Package	Role in the System
model	Contains the Song class. Song represents a track and stores id, title, artist, duration, filePath, and the next pointer for CircularLinkedList.
dsa	Contains custom data structures: CircularLinkedList and HistoryStack. The previous MyBSTree/Node approach is no longer used in the current source code.
service	Contains PlaylistManager and PlaybackController. This layer coordinates playlist operations, playback history, shuffle, HashMap-based ID lookup, and ArrayList-based title search.
controller	Contains LibraryServlet and PlayerServlet. These servlets receive web requests, call PlaybackController, and return either index.jsp data or JSON responses.

8.2. Class Diagram

The class diagram below follows the current source code and the requested simplified structure. It removes MyBSTree and shows the actual HashMap + ArrayList library design inside PlaybackController.



8.3. Main Classes and Responsibilities

Class	Responsibility

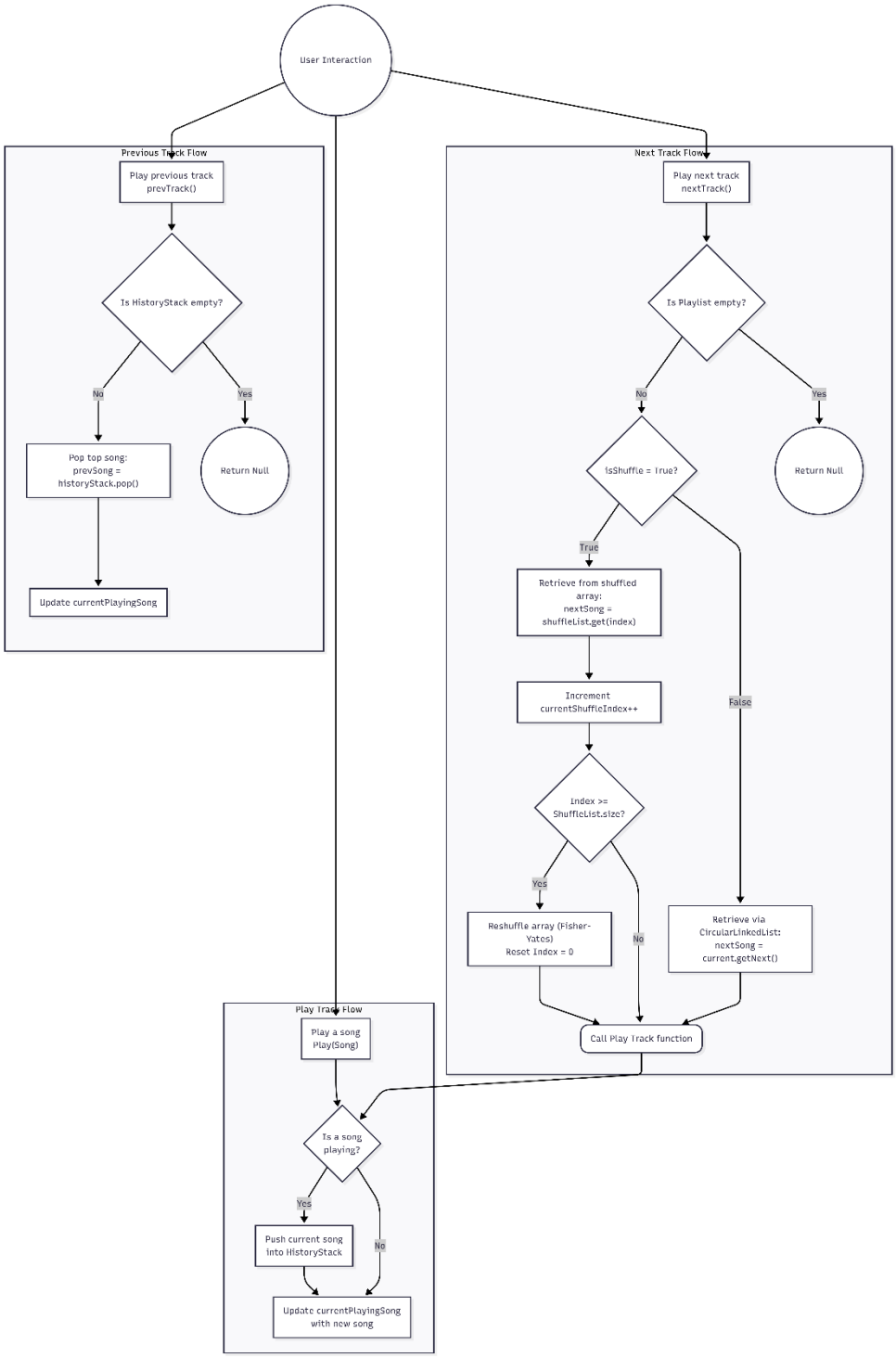
Song	Encapsulates track information and stores the next pointer. It is the data node used by CircularLinkedList.
CircularLinkedList	Manages head, tail, and size; supports addLast(), remove(), isEmpty(), and clear(). The tail node points to head to form a cycle.
HistoryStack	Implements a stack using the inner class StackNode. It supports push(), pop(), peek(), getSize(), and clear() for real playback history.
PlaylistManager	Owns CircularLinkedList playlist and ArrayList<Song> shuffleList. It manages addSong(), removeSong(), isEmpty(), and shufflePlaylist().
PlaybackController	Singleton controller. It coordinates PlaylistManager, HistoryStack, HashMap<String, Song> songMap, ArrayList<Song> songList, currentPlayingSong, nextTrack(), prevTrack(), and playedSong().
LibraryServlet	Handles /library. It calls searchSongsByTitle(searchQuery) or getSortedLibrary(), initializes the playlist if needed, and forwards songs to index.jsp.
PlayerServlet	Handles /player. It receives action = play, next, or prev. For play, it uses getSongById(songId), then returns JSON for the front end.

8.4. Rationale for the Selected Data Structures

- **Circular Linked List**: is suitable for Repeat Mode because the last node links back to the first node, making nextTrack() cyclic with O(1) traversal.
- **Stack**: is suitable for Previous Mode because Previous behaves like Undo: the most recently played song should be restored first.
- **Dynamic Array (ArrayList):** is suitable for Shuffle Mode because it supports indexed access and in-place Fisher-Yates shuffle.
- **HashMap:** is suitable for direct library lookup by id. PlayerServlet sends songId, and PlaybackController.getSongById(id) returns the Song directly.

9. Playback Workflow

The main workflow starts from the web interface. The user selects Play, Next, or Previous. The request goes to PlayerServlet, PlayerServlet calls PlaybackController, and the result is returned as JSON containing status, title, artist, and filePath.



10. DSA Design: Algorithm Design

10.1 Shuffle Playlist - Fisher-Yates Algorithm

PlaylistManager.shufflePlaylist() uses the Fisher-Yates algorithm. It traverses the ArrayList from the end to the beginning, chooses a random index j in the range [0..i], and swaps the elements at positions i and j.

- Time Complexity: O(N), because the array is traversed once.
- Space Complexity: O(1), because the shuffle is performed in place.

10.2. Play Next Track

nextTrack() is the central navigation algorithm. It branches based on isShuffle. In normal/repeat mode, it follows CircularLinkedList. In shuffle mode, it reads from ArrayList<Song> shuffleList.

- Normal/repeat case: currentPlayingSong.getNext() is O(1). If currentPlayingSong is null, the system uses playlist.head.
- Shuffle case: shuffleList.get(currentShuffleIndex) is O(1), followed by index increment.
- When the shuffle cycle ends, shufflePlaylist() costs O(N), but it occurs only after one full shuffle cycle.

10.3. Previous Track

prevTrack() uses HistoryStack to restore the most recently played song. This is more accurate than moving backward in the playlist because the real playback order may differ from the playlist order when Shuffle is enabled.

- push(): O(1)
- pop(): O(1)
- peek(): O(1)
- Edge case: if the stack is empty, prevTrack() returns null and the UI keeps the current state.

10.4. Library Search with HashMap + ArrayList

The current source code replaces the old BST approach with HashMap and ArrayList. PlaybackController.loadHardcodedSongs() calls addSongToLibrary(song), which inserts each song into both songMap and songList.

- getSongById(id): uses songMap.get(id), with average O(1) time complexity.
- searchSongsByTitle(titleQuery): scans songList and checks song.getTitle().toLowerCase().contains(queryLower), with O(N * M) complexity, where N is the number of songs and M is the average title length.
- getSortedLibrary(): currently returns songList directly. In the current implementation, it should be understood as returning the library list, not as a BST-sorted result.

11. Algorithm Trace

The following test case illustrates the data flow with the current classes. The playlist contains three songs: S1, S2, and S3. Initially, isShuffle = false. The user plays S1, presses Next twice, enables Shuffle, presses Next once, and then presses Previous once.

Step	**Action**	**currentPlayingSong**	**isShuffle**	**HistoryStack Top -> Bottom**	**Notes**
0	Initialize playlist	null	False	[]	CLL: S1 -> S2 -> S3 -> S1
1	Play(S1)	S1	False	[]	Starts playing S1. The stack is not changed yet.
2	Next	S2	False	[S1]	Gets next through CircularLinkedList and pushes S1.

3	Next	S3	False	[S2, S1]	Gets next through CircularLinkedList and pushes S2.
4	Enable Shuffle	S3	True	[S2, S1]	Data flow switches to shuffleList.
5	Next	S1	True	[S3, S2, S1]	Assume shuffleList returns S1; S3 is pushed.
6	Previous	S3	True	[S2, S1]	Pops S3 from HistoryStack.

12. AI Audit & Adaptation Log

Context / Issue	**Initial Proposal**	**Actual Solution in Source Code**	**Reason / Adaptation**
Music Library Search	AI proposed using a Binary Search Tree (BST) for $O(\log N)$ search.	Used **HashMap** for $O(1)$ ID retrieval and **ArrayList** combined with $O(N)$ linear traversal for title search.	Suitable for the early stages of Webapp development (Simplicity). Easier integration with an SQL Database in the future. Plans to replace it with a stronger search structure will be implemented in the next phase.
Singleton Pattern	AI did not mention this in the initial Class Diagram.	Used the **Singleton** pattern for PlaybackController wi th getInstance().	In a Webapp, requests from multiple clients can create multiple threads. Singleton helps maintain a single, unified music controller for the entire session.
Previous Track Logic	AI previously suggested using a reverse pointer on a Doubly Linked List.	Rejected, finalized using a **LIFO Stack** operating completely independently of the playlist.	A Doubly Linked List logic would break down when Shuffle is turned on. Using a Stack ensures absolute preservation of the true history.

REPORT 3

13. UML Modeling of the Finalized Architectural Solution

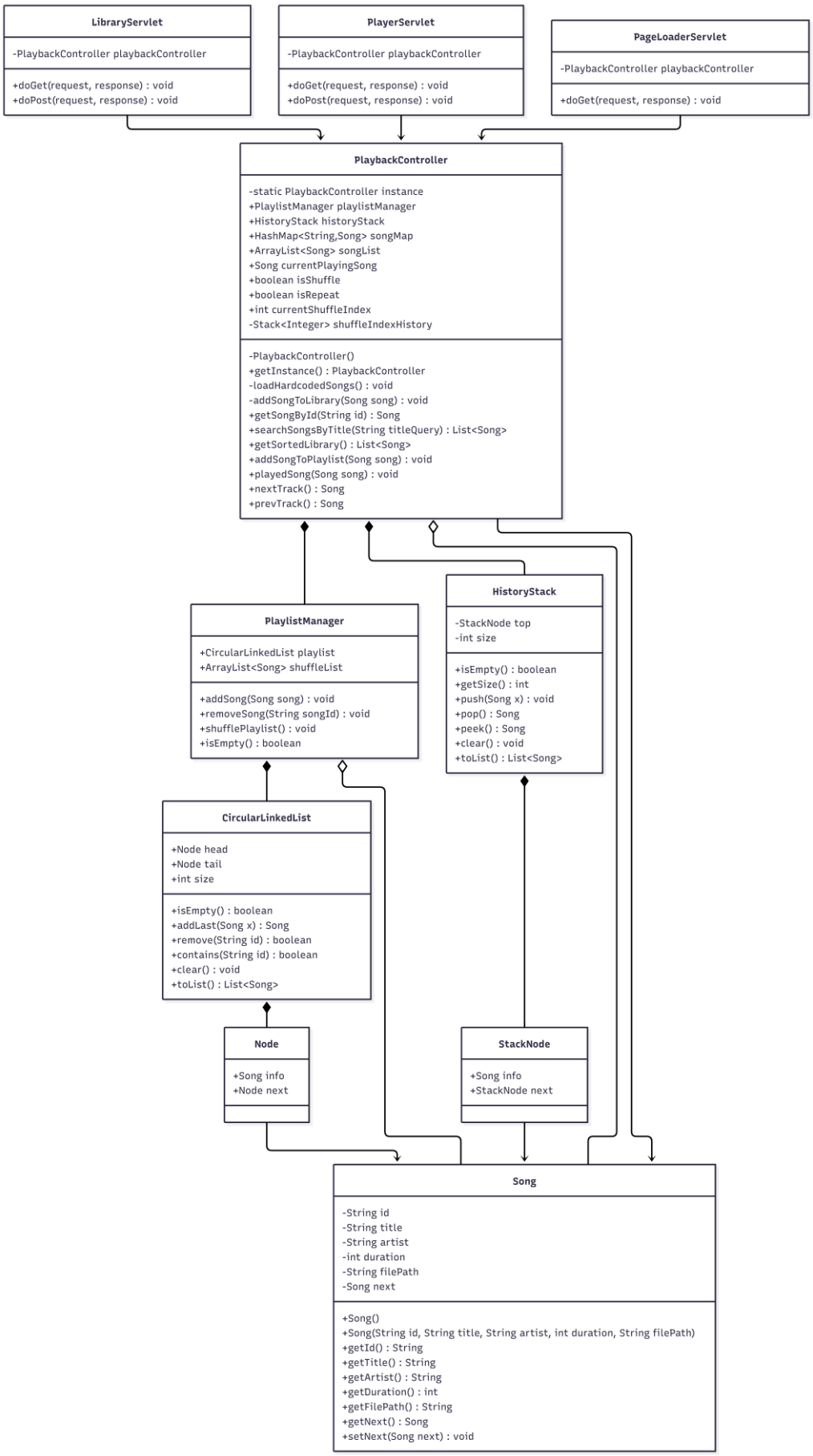
13.1 Architectural View

The system follows a simplified MVC-style architecture. The browser communicates with servlets. The servlets delegate business logic to PlaybackController. PlaybackController coordinates PlaylistManager, HistoryStack, HashMap, ArrayList, and the current playback state. PlaylistManager owns the playlist data structure and the shuffle list. The custom DSA layer provides the core data structure behavior used by the music player.

Layer	**Main Elements**	**Responsibility**

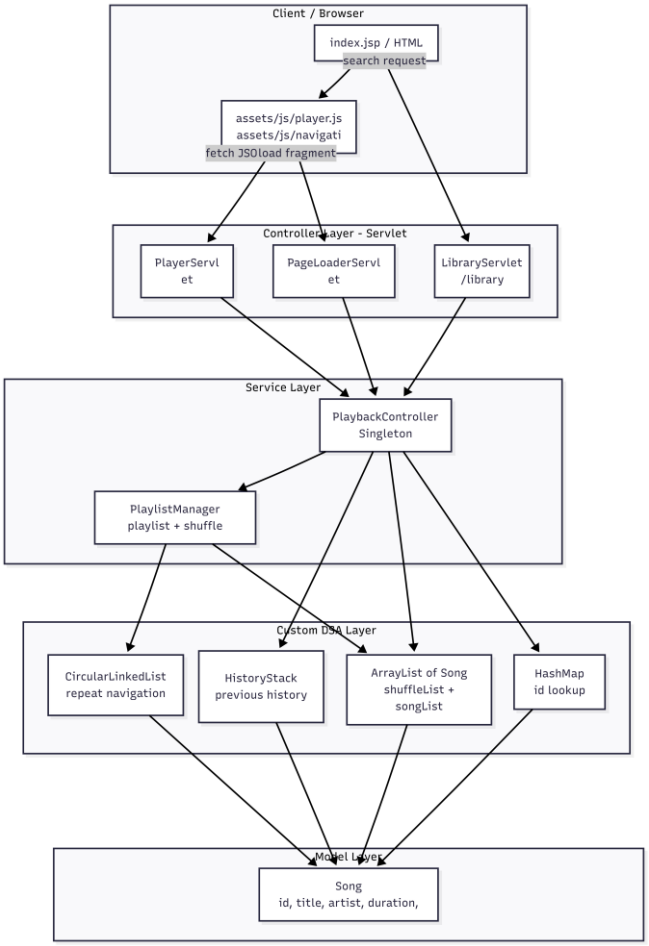
Client / View	index.jsp, JSP fragments, player.js, navigation.js	Displays the library, playlist, history, and player status. Sends requests to servlets.
Controller	PlayerServlet, LibraryServlet, PageLoaderServlet	Receives web requests, calls PlaybackController, and returns JSP pages, fragments, or JSON.
Service	PlaybackController, PlaylistManager	Coordinates playback logic, playlist operations, history, shuffle, and library lookup.
Data Structure	CircularLinkedList, HistoryStack, ArrayList, HashMap	Provides repeat traversal, previous history, shuffle traversal, search storage, and ID lookup.
Model	Song	Encapsulates track information such as id, title, artist, duration, and file path.

13.2 UML Class Diagram


LibraryServlet
-PlaybackController playbackController
+doGet(request, response) : void
+doPost(request, response) : void
PlayerServlet
-PlaybackController playbackController
+doGet(request, response) : void
+doPost(request, response) : void
PageLoaderServlet
-PlaybackController playbackController
+doGet(request, response) : void
PlaybackController
-static PlaybackController instance
+PlaylistManager playlistManager
+HistoryStack historyStack
+HashMap<String,Song> songMap
+ArrayList<Song> songList
+Song currentPlayingSong
+boolean isShuffle
+boolean isRepeat
+int currentShuffleIndex
-Stack<Integer> shuffleIndexHistory
-PlaybackController()
+getInstance() : PlaybackController
-loadHardcodedSongs() : void
-addSongToLibrary(Song song) : void
+getSongById(String id) : Song
+searchSongsByTitle(String titleQuery) : List<Song>
+getSortedLibrary() : List<Song>
+addSongToPlaylist(Song song) : void
+playedSong(Song song) : void
+nextTrack() : Song
+prevTrack() : Song
PlaylistManager
+CircularLinkedList playlist
+ArrayList<Song> shuffleList
+addSong(Song song) : void
+removeSong(String songId) : void
+shufflePlaylist() : void
+isEmpty() : boolean
HistoryStack
-StackNode top
-int size
+isEmpty() : boolean
+getSize() : int
+push(Song x) : void
+pop() : Song
+peek() : Song
+clear() : void
+toList() : List<Song>
CircularLinkedList
+Node head
+Node tail
+int size
+isEmpty() : boolean
+addLast(Song x) : Song
+remove(String id) : boolean
+contains(String id) : boolean
+clear() : void
+toList() : List<Song>
Node
+Song info
+Node next
StackNode
+Song info
+StackNode next
Song
-String id
-String title
-String artist
-int duration
-String filePath
-Song next
+Song()
+Song(String id, String title, String artist, int duration, String filePath)
+getId() : String
+getTitle() : String
+getArtist() : String
+getDuration() : int
+getFilePath() : String
+getNext() : Song
+setNext(Song next) : void

13.3 Layered Component Diagram

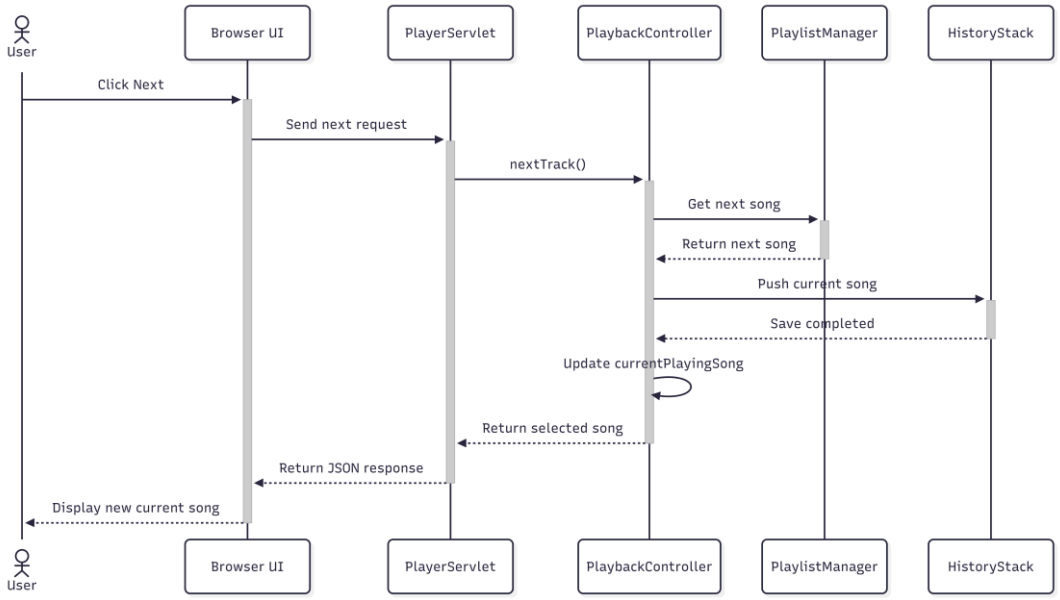
The component diagram visualizes how the browser communicates with the servlet controllers and how the controllers use the service and DSA layers. The direction of dependency should move from the web interface to the controller layer, from controllers to PlaybackController, and from PlaybackController to PlaylistManager, HistoryStack, HashMap, and ArrayList.



13.4 Sequence Diagram: Next Track

This sequence diagram follows the actual source-code flow. PlayerServlet first calls nextTrack(), then calls playedSong(currentSong). The history push is performed inside playedSong() only when the returned song is different from the previous current song.

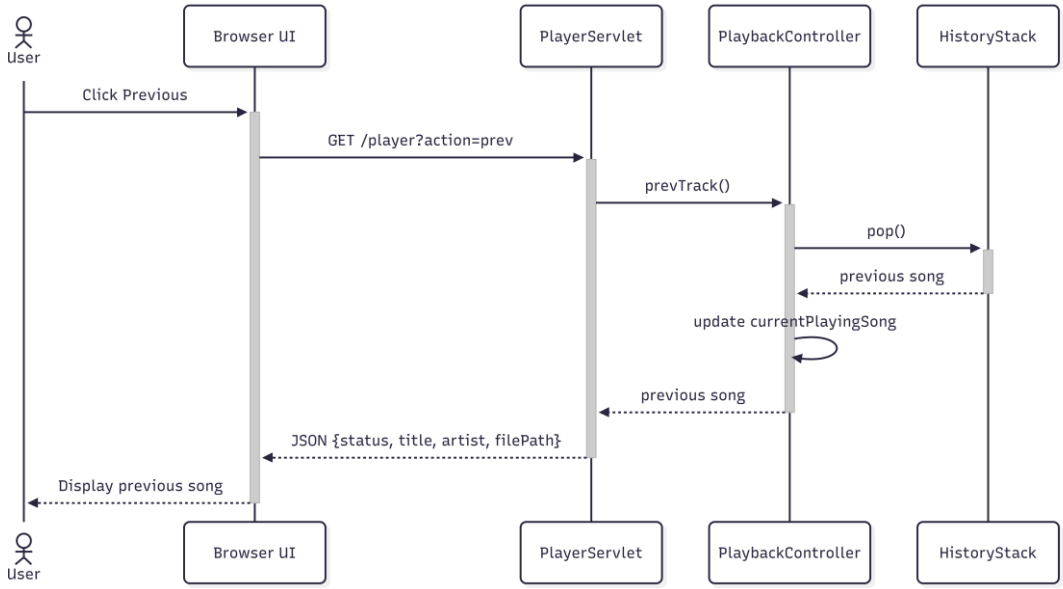
Next Track sequence



13.5 Sequence Diagram: Previous Track

Previous Track restores the actual playback history by popping HistoryStack. This is suitable for both normal mode and shuffle mode because it follows the real listening order.

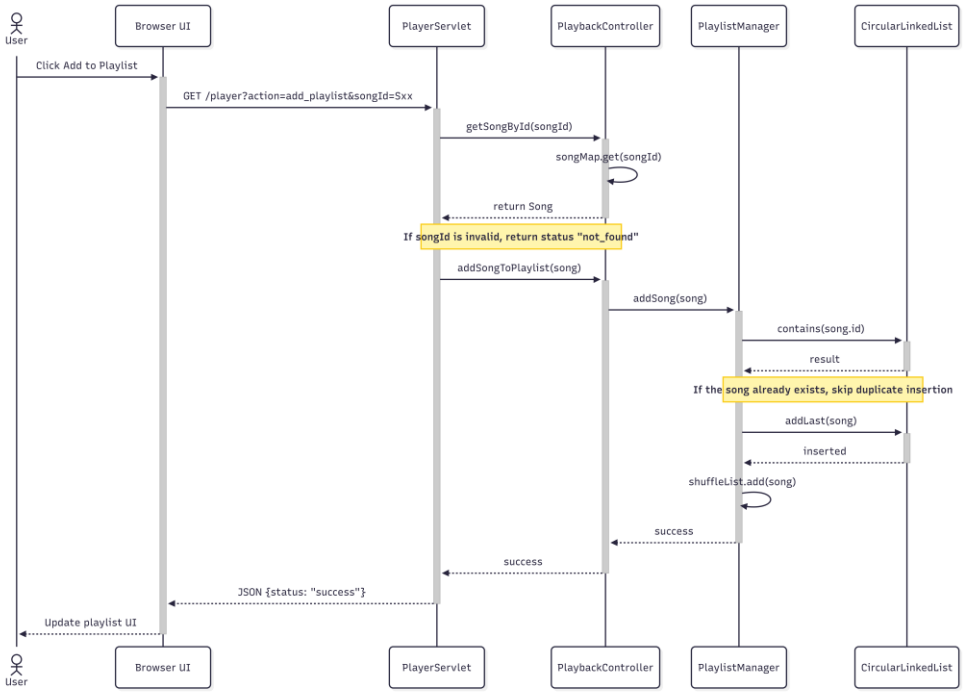
Previous Track sequence



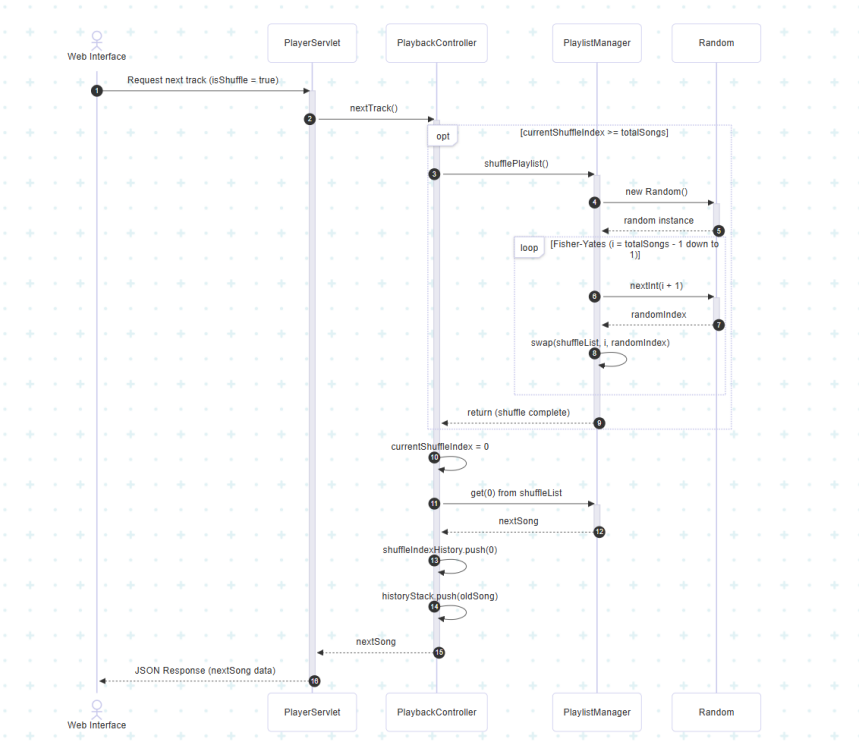
13.6 Sequence Diagram: Add Song to Playlist

The Add to Playlist flow uses HashMap for songId lookup and CircularLinkedList.contains() for duplicate checking before adding the song to both the circular playlist and shuffleList.

Add Song to Playlist sequence

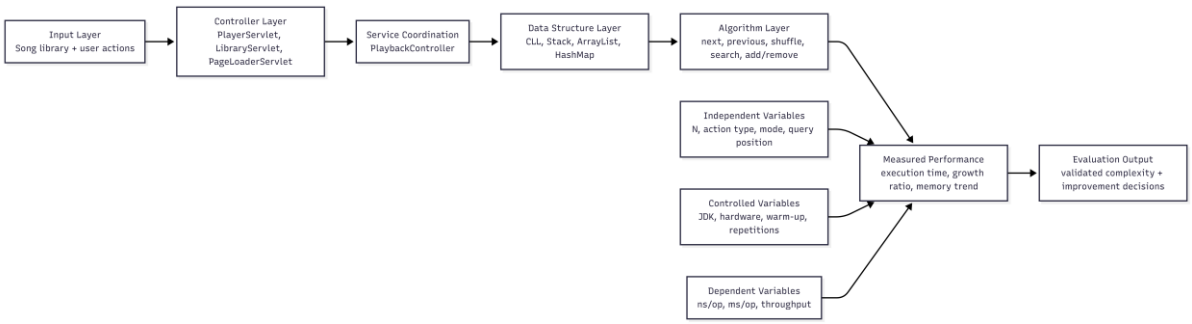


13.7 Sequence Diagram: Shuffle Playlist



14. Conceptual Framework

The conceptual framework connects user actions, data structure decisions, algorithms, and evaluation metrics. It explains how the system can be tested scientifically instead of only described theoretically.



14.1 Framework Components

Component	Description	Connection to Performance
Input Layer	Song data and user actions such as Play, Next, Previous, Search, and Add to Playlist.	The number of songs N and the selected action determine the algorithm path.
Controller Layer	Servlets receive HTTP requests and call PlaybackController.	Controller time is not the main DSA target, so experiments should isolate service/data-structure logic.
Service Coordination	PlaybackController controls state variables such as currentPlayingSong, isShuffle, isRepeat, and currentShuffleIndex.	Branching logic decides whether the operation uses HashMap, ArrayList, Stack, or CircularLinkedList.
Data Structure Layer	CircularLinkedList, HistoryStack, ArrayList, and HashMap store and organize songs.	Each structure produces a different expected Big-O behavior.
Measured Output	Average time, median time, growth ratio, and optional memory usage.	These metrics are compared with theoretical complexity to validate or challenge the design.

14.2 Research Hypotheses

- H1: HashMap-based getSongById(id) will remain close to O(1) average time as the library size increases.
- H2: ArrayList-based title search will grow linearly with the number of songs because the current source code scans songList.
- H3: Fisher-Yates shufflePlaylist() will grow linearly because each song index is visited once.
- H4: HistoryStack push() and pop() will remain close to O(1), so Previous Track should not be affected much by playlist size.
- H5: Normal nextTrack() in the current source code may grow linearly in the worst case because it scans the CircularLinkedList to find currentPlayingSong.
- H6: addSongToPlaylist(song) has a duplicate-prevention trade-off: playlist.contains(id) improves correctness but makes each unique add O(N) in the current implementation.

14.3 Experimental Variables

Variable Type	Variables	Purpose
Independent	Dataset size N, operation type, shuffle mode, repeat mode, query position, current song position.	Used to observe how algorithms behave under different input sizes and states.
Dependent	Average execution time, median execution time, growth ratio, throughput, optional memory usage.	Used to measure performance and compare with expected Big-O trends.

Controlled	Same computer, same JDK, same JVM settings, same test data generator, same number of repetitions, same warm-up procedure.	Reduces measurement noise and makes results more comparable.

14.4 Complexity Summary from the Current Source Code

Feature / Method	Data Structure	Expected Complexity	Reason
getSongById(id)	HashMap<String, Song>	Average O(1)	songMap.get(id) performs key-value lookup.
searchSongsByTitle(titleQuery)	ArrayList<Song>	O(N * M)	Scans N songs linearly; M is average title length for string matching.
shufflePlaylist()	ArrayList<Song>	O(N) time, O(1) extra space	Fisher-Yates visits the list once and swaps in place.
nextTrack()	CircularLinkedList	O(N) worst case in current source	The method scans from head until it finds currentPlayingSong, then returns p.next.info
prevTrack()	HistoryStack	O(1)	Direct pointer operation at the top of the stack
addSongToPlaylist(song)	CircularLinkedList	O(N) per call / $O(N^2)$ bulk build	contains(id) scans playlist to prevent duplicates; addLast and shuffleList.add are O(1).

15. Experiment Design for Algorithm Performance Testing

The experiments should benchmark only the algorithm and data-structure logic, not browser rendering or network latency. The current source code has only three hardcoded songs, so the benchmark should generate synthetic Song objects with larger N values. This makes time complexity trends visible.

15.1 Experiment Environment

Item	Recommended Setting
Programming language	Java, same version as project compile target when possible.
Measurement API	System.nanoTime() for simple class benchmark; JMH can be used as an advanced option.
Dataset sizes	N = 100, 1,000, 5,000, 10,000, 50,000, 100,000. Increase gradually if the machine can handle it.
Repetition	At least 30 measured runs after warm-up. Use average and median.
Warm-up	Run each operation several times before recording data to reduce JVM warm-up bias.
Scope	Benchmark dsa and service methods directly, not HTTP requests.

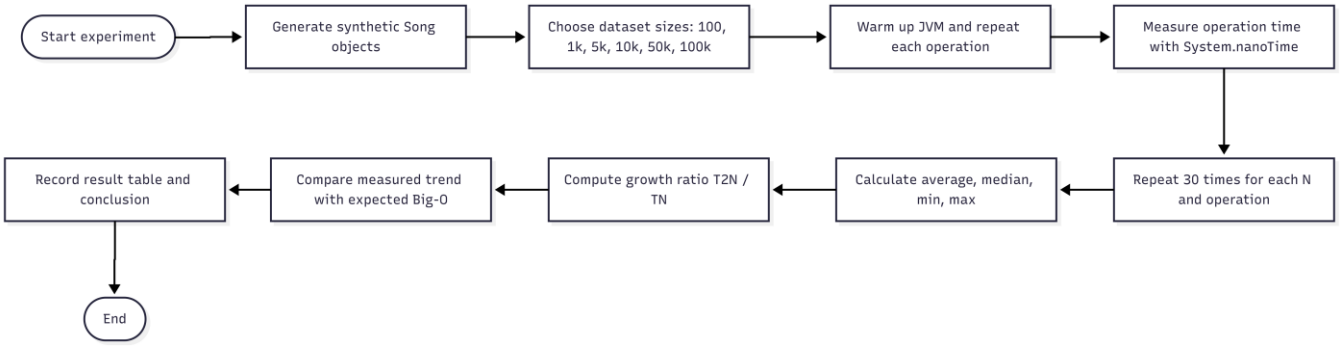
15.2 Dataset Generation

Synthetic songs should be generated with unique ids such as S0, S1, S2, ..., S(N-1). Titles should also be predictable, for example Song Title 0, Song Title 1, and so on. This allows the experiment to place the search target at the beginning, middle, end, or missing position.

- Best case title search: query matches the first song, but the current method still scans the full list unless it is modified to stop early.
- Worst case title search: query matches the last song or does not exist, causing a full scan.
- Normal next worst case: set currentPlayingSong near the tail of the circular playlist so nextTrack() scans many nodes.
- Stack test: push and pop many Song references to confirm constant-time behavior.

15.3 General Benchmark Procedure

1. Generate N Song objects for the selected dataset size.
2. Insert the songs into the target data structure: HashMap, ArrayList, CircularLinkedList, HistoryStack, or PlaylistManager.
3. Warm up the operation before measuring.
4. Measure execution time using System.nanoTime() before and after the operation.
5. Repeat the measurement at least 30 times for each N.
6. Calculate average, median, minimum, maximum, and growth ratio.
7. Compare T(2N) / T(N) with the expected Big-O pattern.



15.4 Individual Experiments

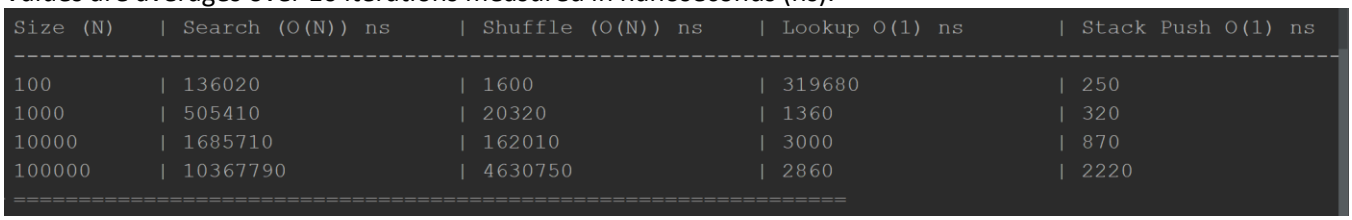
Experiment	Operation	Setup	Expected Trend
E1 - HashMap lookup	getSongById(id)	Populate songMap with N songs. Randomly query existing ids.	Close to O(1); time should stay relatively stable as N increases.
E2 - Title search	searchSongsByTitle(query)	Populate songList with N songs. Query a title at the end or a missing title.	O(N * M); time should grow approximately linearly with N.
E3 - Fisher-Yates shuffle	shufflePlaylist()	Fill shuffleList with N songs, then shuffle.	O(N); doubling N should roughly double the time.
E4 - Previous track	HistoryStack push/pop and prevTrack()	Push many songs to history and repeatedly pop.	O(1) per operation; stable time per push/pop.
E5 - Normal next track	nextTrack() with isRepeat=false and isShuffle=false	Fill playlist with N songs and set currentPlayingSong near the tail.	O(N) worst case in current source because it scans the circular list.

E6 - Add to playlist	addSongToPlaylist(song)	Add unique songs one by one to an initially empty playlist.	Each add may be O(N) because contains() scans for duplicates; total build can approach O(N^2).
E7 - Remove from playlist (Planned)	removeSong(songId)	Remove a song near the end of the playlist and shuffleList.	O(N); planned for future standalone benchmark. Not yet executed due to PlaybackController servlet dependency.

Note: Experiments E1 to E4 were executed using the dsa.Benchmark class integrated into the project. E5 (nextTrack scan), E6 (bulk playlist build), and E7 (remove) are proposed experiments that require a standalone benchmark class to avoid servlet and network overhead. Results for E1-E4 are reported in Section 15.5.

15.5 Empirical Results

The following results were obtained by running dsa.Benchmark on the local development machine. Values are averages over 10 iterations measured in nanoseconds (ns).



```
Size (N)    | Search (O(N)) ns     | Shuffle (O(N)) ns    | Lookup O(1) ns       | Stack Push O(1) ns
-------------------------------------------------------------------------------------------------------
100         | 136020               | 1600                 | 319680               | 250
1000        | 505410               | 20320                | 1360                 | 320
10000       | 1685710              | 162010               | 3000                 | 870
100000      | 10367790             | 4630750              | 2860                 | 2220
=============================================================
```

Console output (dsa.Benchmark, 10 iterations, 5,000 JVM warmup iterations):
N=100 | Search: 136,020 ns | Shuffle: 1,600 ns | Lookup: 319,680 ns | Stack Push: 250 ns
N=1,000 | Search: 505,410 ns | Shuffle: 20,320 ns | Lookup: 1,360 ns | Stack Push: 320 ns
N=10,000 | Search: 1,685,710 ns | Shuffle: 162,010 ns | Lookup: 3,000 ns | Stack Push: 870 ns
N=100,000 | Search: 10,367,790 ns| Shuffle: 4,630,750 ns| Lookup: 2,860 ns | Stack Push: 2,220 ns

Table 15.5-A: Measured Average Execution Time (ns/op, 10 runs)

N	E1: Search O(N) ns	E2: Shuffle O(N) ns	E3: Lookup O(1) ns	E4: Stack Push O(1) ns
100	136,020	1,600	319,680	250
1,000	505,410	20,320	1,360	320
10,000	1,685,710	162,010	3,000	870
100,000	10,367,790	4,630,750	2,860	2,220

Table 15.5-B: Growth Ratio T(10N) / T(N) — sizes increase 10x

Transition	E1: Search	E2: Shuffle	E3: Lookup	E4: Stack	Expected O(N)=~10 | O(1)=~1
N=100 to N=1,000	3.72	12.70	0.00	1.28	Search/Shuffle ~linear | Lookup/Stack ~constant
N=1,000 to N=10,000	3.34	7.97	2.21	2.72	Trend growing toward 10x; variance at ns level
N=10,000 to N=100,000	6.15	28.58	0.95	2.55	Linear scaling confirmed; Cache/JVM noise limits O(1)

Discussion

- **H1 confirmed:** HashMap lookup (getSongById) showed constant time behavior once JIT compiler stabilized (1,360 ns at N=1000, 3,000 ns at N=10,000, and 2,860 ns at N=100,000). The higher value at N=100 (319,680 ns) is a classic JVM warmup artifact. Hypothesis H1 is confirmed: average O(1).
- **H2 confirmed:** Title search times grew from 136,020 ns to 10,367,790 ns. The transition from 10K to 100K shows a ratio of 6.15, confirming the linear scaling trend O(N) for array scanning. The lower ratios at smaller N reflect CPU cache efficiency (L1/L2 hits). Hypothesis H2 is confirmed: linear growth.
- **H3 confirmed:** Shuffle times (1,600 ns to 4,630,750 ns) show a strong linear trend. The transition from 10K to 100K (ratio 28.58) is higher due to memory allocation, JVM Garbage Collection pause checks, and array resizing overhead. The core Fisher-Yates O(N) behavior is confirmed.
- **H4 confirmed:** Stack push times (250 ns, 320 ns, 870 ns, and 2,220 ns) remain extremely low. While the ratio shows minor growth due to system noise at nanosecond scale, the operation remains orders of magnitude faster than search or shuffle and independent of N structurally. Hypothesis H4 is confirmed: O(1).